

Flutter Fundamentals

Giới thiệu Flutter

Contents

1	Giới thiệu chung về Flutter	2
1.1	Thiết lập môi trường phát triển Flutter	2
	Bài tập 1. Cài đặt môi trường phát triển ứng dụng với Flutter	2
	Bài tập 2. Kiểm tra phiên bản Flutter và Dart	2
	Bài tập 3. Kiểm tra thư mục Flutter SDK nằm ở đâu?	2
	Bài tập 4. Nâng cấp phiên bản Flutter.....	4
	Bài tập 5. Xử lý khi máy cài nhiều phiên bản Flutter SDK	5
1.2	Tạo ứng dụng Flutter đầu tiên	7
	Bài tập 6. Ứng dụng Flutter đầu tiên – Hello World	7
	Bài tập 7. Xem ứng dụng ở các kích thước màn hình khác nhau (responsive)	9
1.3	Cấu trúc project Flutter	11
	Bài tập 8. Tìm hiểu chi tiết về cấu trúc project Flutter	12
	Bài tập 9. Giải thích file main.dart	17
1.4	Một số bài thực hành làm quen với Flutter	22
	Bài tập 10. Áp dụng kiểu (style) cho widget Text.....	22
	Bài tập 11. Hiển thị hình chữ nhật với widget Container	23
	Bài tập 12. Hiển thị hình tròn với widget Container.....	24
	Bài tập 13. Hiển thị icon	25
	Bài tập 14. Hiển thị ảnh từ asset với Image.asset (ảnh được đóng gói trong app)	28
	Bài tập 15. Hiển thị ảnh từ internet với Image.network.....	29
	Bài tập 16. Hiển thị một nút (button) trong Flutter	30
	Bài tập 17. Tạo bộ khung cho màn hình với widget Scaffold.....	31
1.5	Troubleshooting	32
1.5.1	Error: Null safety features are disabled for this library	32

1 Giới thiệu chung về Flutter

1.1 Thiết lập môi trường phát triển Flutter

Bài tập 1. Cài đặt môi trường phát triển ứng dụng với Flutter

Tham khảo: <https://docs.flutter.dev/install>

Chọn một trong hai cách cài đặt sau:

- Cài đặt thông qua VS Code (khuyến khích):
<https://docs.flutter.dev/install/with-vs-code>
- Cài đặt thủ công: <https://docs.flutter.dev/install/manual>

Bài tập 2. Kiểm tra phiên bản Flutter và Dart

Yêu cầu:

Dùng lệnh gì để kiểm tra phiên bản Flutter và Dart đang được cài trên máy?

Hướng dẫn:

Để kiểm tra phiên bản Flutter SDK: `flutter --version`

Để kiểm tra phiên bản ngôn ngữ Dart: `dart --version`

Ví dụ:

```
C:\Users\Admin>flutter --version
```

```
A new version of Flutter is available!
```

```
To update to the latest version, run "flutter upgrade".
```

```
Flutter 3.24.5 • channel stable • https://github.com/flutter/flutter.git
Framework • revision dec2ee5c1f (10 months ago) • 2024-11-13 11:13:06 -0800
Engine • revision a18df97ca5
Tools • Dart 3.5.4 • DevTools 2.37.3
```

Bài tập 3. Kiểm tra thư mục Flutter SDK nằm ở đâu?

Yêu cầu: Hãy kiểm tra thư mục Flutter SDK đang dùng hiện tại ở vị trí nào?

Cách 1: Dùng lệnh flutter doctor

Mở **Terminal/Command line** > gõ lệnh `flutter doctor -v`

Ví dụ:

```
C:\Users\Admin>flutter doctor -v
[✓] Flutter (Channel stable, 3.35.4, on Microsoft Windows [Version
10.0.19045.6218], locale en-US) [777ms]
    • Flutter version 3.35.4 on channel stable at C:\flutter
    • Upstream repository https://github.com/flutter/flutter.git
    • Framework revision d693b4b9db (5 days ago), 2025-09-16 14:27:41 +0000
    • Engine revision c298091351
    • Dart version 3.9.2
    • DevTools version 2.48.0
    • Feature flags: enable-web, enable-linux-desktop, enable-macos-desktop,
enable-windows-desktop, enable-android,
enable-ios, cli-animations, enable-lldb-debugging
```

Cách 2: Dùng lệnh `where flutter` (Windows) hoặc `which flutter` (Mac/Linux)

Ví dụ:

```
C:\Users\Admin>where flutter
C:\flutter\bin\flutter
C:\flutter\bin\flutter.bat
```

Cách 3: Trong các IDEs:

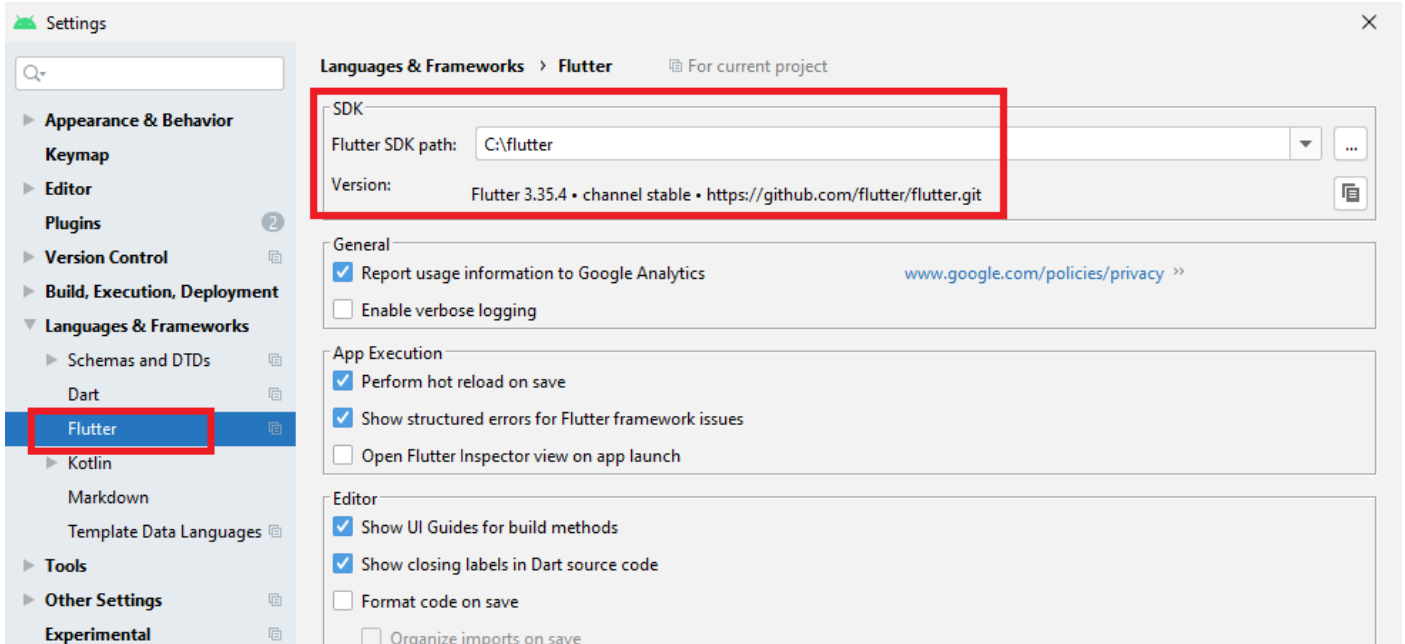
a) Nếu trong VS Code

Vào **View > Terminal**, gõ lệnh: `flutter doctor -v`

b) Nếu trong Android Studio

Vào **File > Settings > Languages & Frameworks > Flutter**.

Ở đó sẽ có mục **Flutter SDK path** hiển thị đường dẫn.



Bài tập 4. Nâng cấp phiên bản Flutter

Hãy thực hiện lệnh nâng cấp phiên bản Flutter.

Hướng dẫn:

Chạy lệnh: `flutter upgrade`

Ví dụ:

```
C:\Users\Admin>flutter upgrade
Upgrading Flutter to 3.35.4 from 3.24.5 in C:\flutter_3.24.5...
Checking Dart SDK version...
Downloading Dart SDK from Flutter engine
c29809135135e262a912cf583b2c90deb9ded610...
Expanding downloaded archive with PowerShell...
Building flutter tool...
Running pub upgrade...
Resolving dependencies...
Downloading packages...
Got dependencies.

Upgrading engine...
Downloading android-arm-profile/windows-x64 tools... 1,543ms
Downloading android-arm-release/windows-x64 tools... 929ms
Downloading android-arm64-profile/windows-x64 tools... 1,607ms
Downloading android-arm64-release/windows-x64 tools... 691ms
```

Bài tập 5. Xử lý khi máy cài nhiều phiên bản Flutter SDK

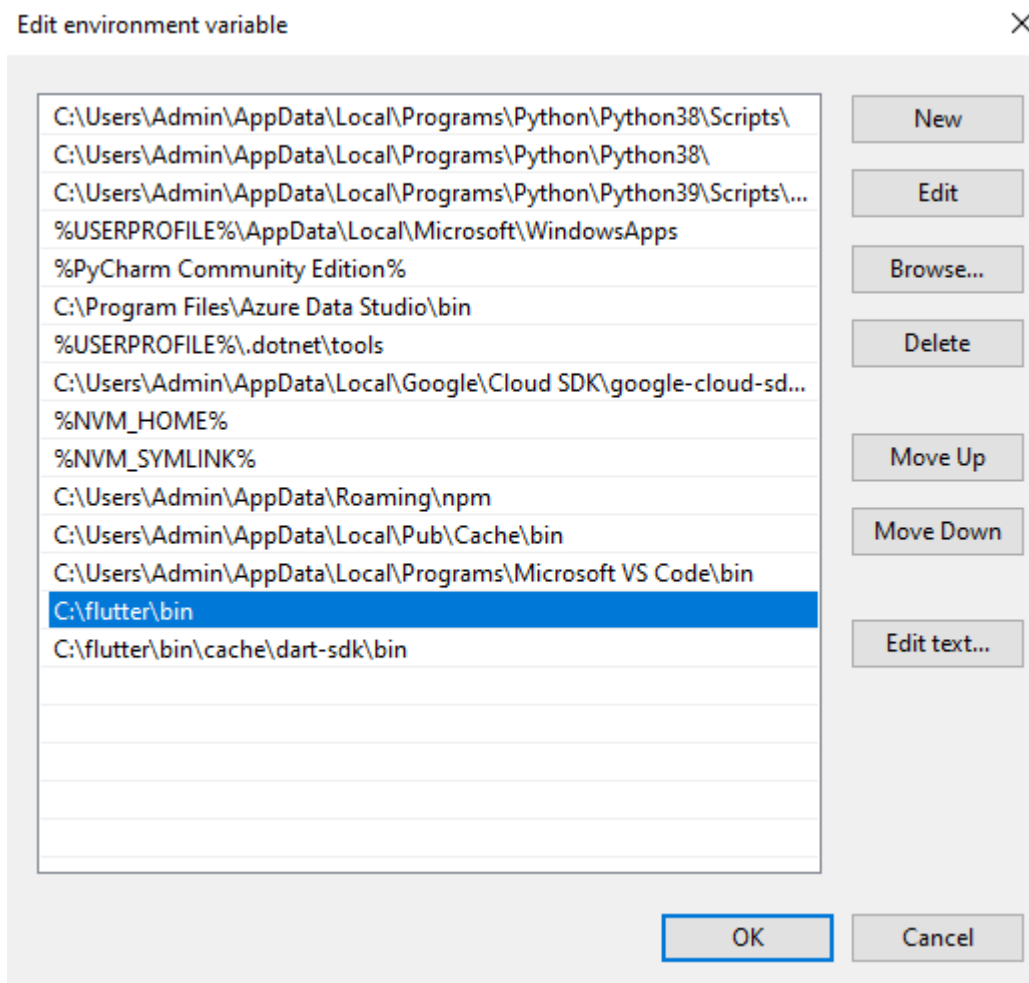
Nếu máy cài nhiều phiên bản Flutter SDK, làm thế nào để ép dùng một phiên bản cụ thể?

Trong máy tính, ta có thể đã cài nhiều phiên bản Flutter SDK, mỗi phiên bản thường được giải nén ở 1 thư mục riêng, ví dụ: `C:\flutter_x.x.x` (với `x.x.x` là số phiên bản).

Vậy làm thế nào để ép dùng 1 phiên bản cụ thể?

Hướng dẫn:

Bước 1: Kiểm tra và cập nhật biến môi trường PATH



Kiểm tra và cập nhật biến môi trường **PATH** để trở vào đúng thư mục **bin** của phiên bản Flutter ta muốn chạy.

Ví dụ: `C:\flutter\bin`

Bước 2: Trong các IDE ta đang dùng (chẳng hạn: Android Studio, Visual Studio Code...), hãy kiểm tra và chỉ định lại đường dẫn đến đúng thư mục Flutter SDK

a) Ép VS Code dùng đúng SDK

Mở VS Code.

Bấm **Ctrl+Shift+P** → gõ `Flutter: Change SDK`.

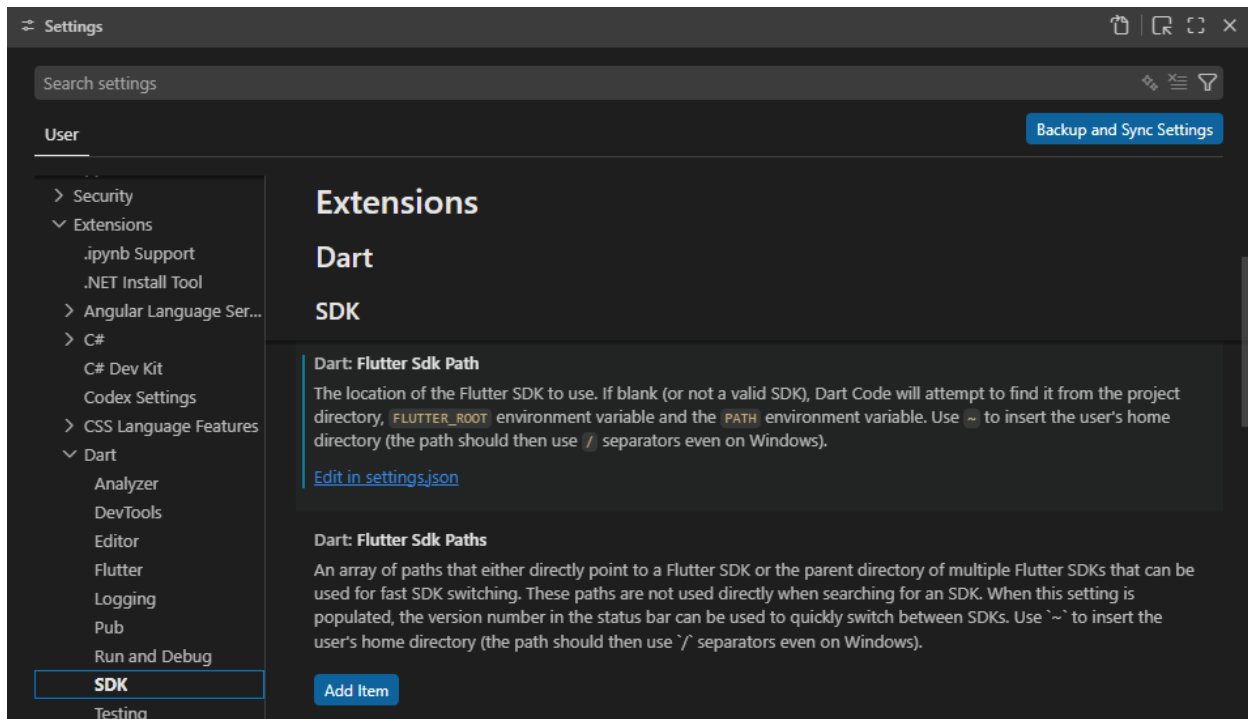
Chọn đường dẫn đến thư mục Flutter SDK (ví dụ: **C:\flutter** hoặc nơi bạn giải nén).

VS Code sẽ reload và dùng SDK mới.

Nếu không thấy, bạn có thể chỉnh thủ công:

Trong VS Code, vào **Settings**, tại hộp thoại **Settings**, vào **Extensions > Dart > SDK**

Tìm đến mục **Dart: Flutter Sdk Path** > **Edit in settings.json**



Mở `settings.json` trong VS Code.

Thêm:

```
"dart.flutterSdkPath": "C:/flutter"
```

b) Ép Android Studio dùng đúng SDK

Mở Android Studio.

Vào menu **File > Settings > Languages & Frameworks > Flutter**.

Trong mục **Flutter SDK path**, chọn đúng đường dẫn Flutter SDK (ví dụ: **C:\flutter**).

Apply → **OK** → **restart Android Studio**.

1.2 Tạo ứng dụng Flutter đầu tiên

Bài tập 6. Ứng dụng Flutter đầu tiên – Hello World

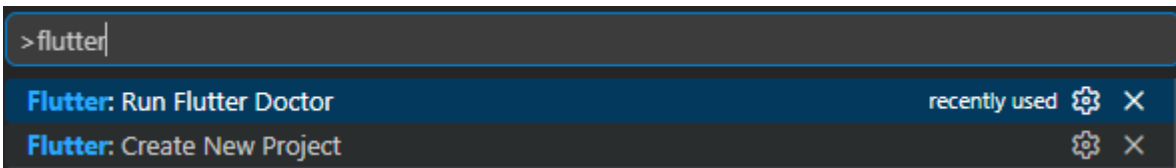
Yêu cầu: Tạo ứng dụng hiển thị chữ: “Hello World”.

Hướng dẫn:

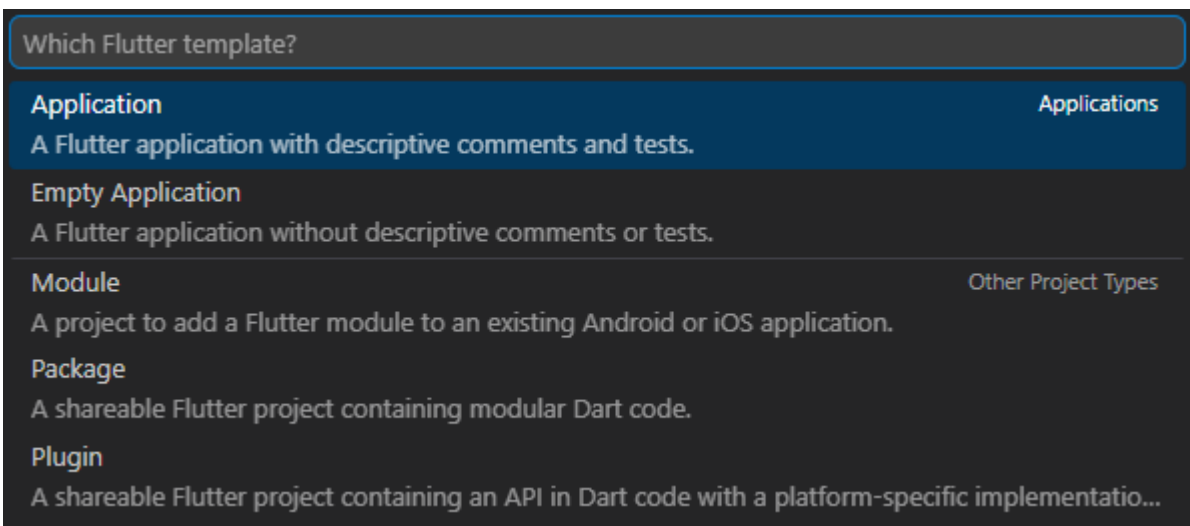
Phần bên dưới minh họa sử dụng VS Code.

Bước 1: Tạo project Flutter mới

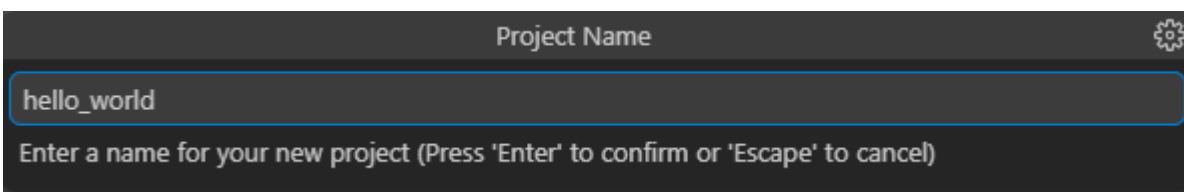
Mở **VS Code** > **View** > **Command Palette** > gõ: **Flutter: Create New Project**



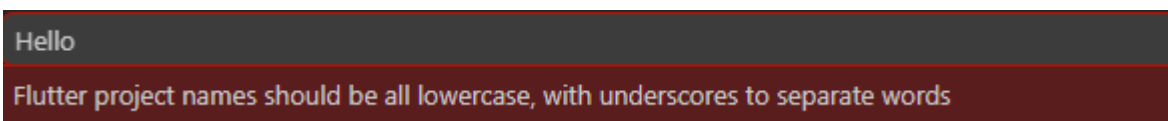
Chọn template: Empty Application



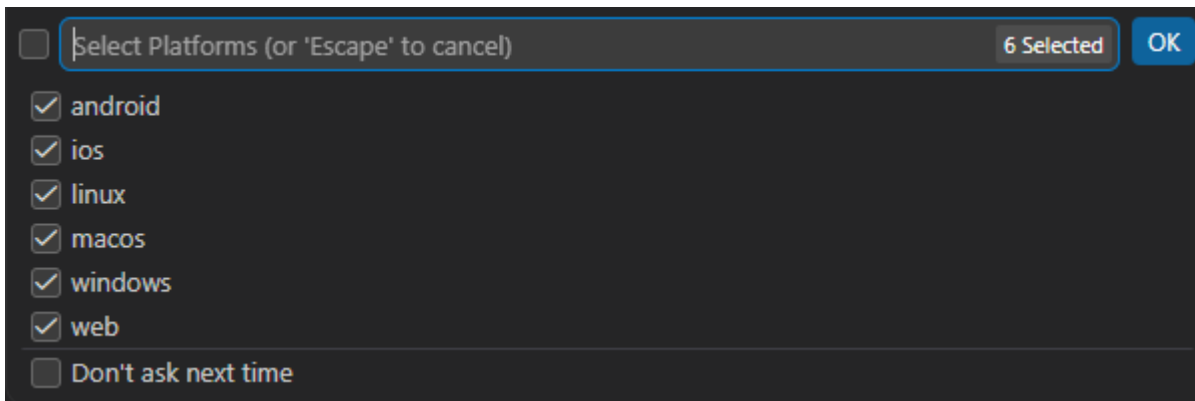
Đặt tên project: hello_world



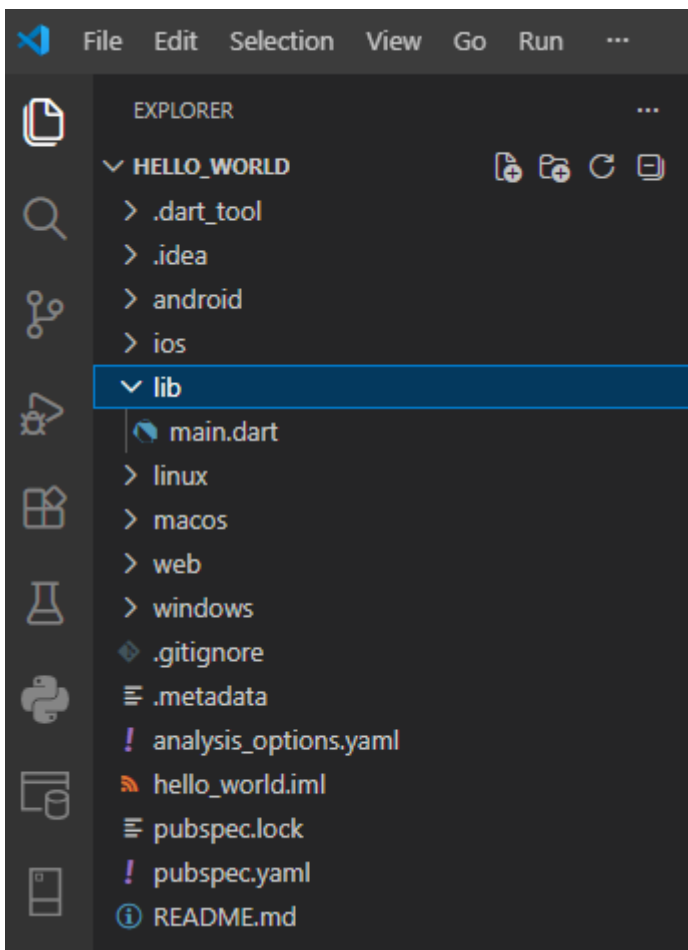
Lưu ý: tên project phải là chữ thường, sử dụng dấu gạch dưới để phân cách các từ.



Chọn Platforms: ta chọn tất cả > Bấm OK.



Đợi một lát, project Flutter được tạo ra:



File lib/main.dart chứa code ứng dụng (như bên dưới)

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});
```

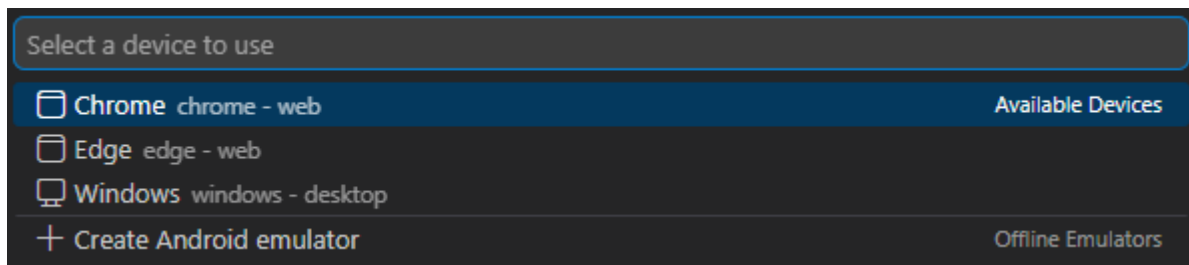
```
@override
Widget build(BuildContext context) {
  return const MaterialApp(
    home: Scaffold(body: Center(child: Text('Hello World!'))),
  );
}
```

Bước 2: Chạy ứng dụng Flutter

Cách 1: Dùng nút lệnh trên VS Code

Vào Run > Start Debugging (hoặc Run Without Debugging)

Chọn thiết bị chạy: ví dụ chọn Chrome



Trình duyệt Chrome được mở ra và hiển thị ứng dụng Flutter.

Cách 2: Dùng lệnh trên Terminal

```
flutter run -d chrome
```

Lệnh trên chạy ứng dụng Flutter trên trình duyệt Chrome.

Bài tập 7. Xem ứng dụng ở các kích thước màn hình khác nhau (responsive)

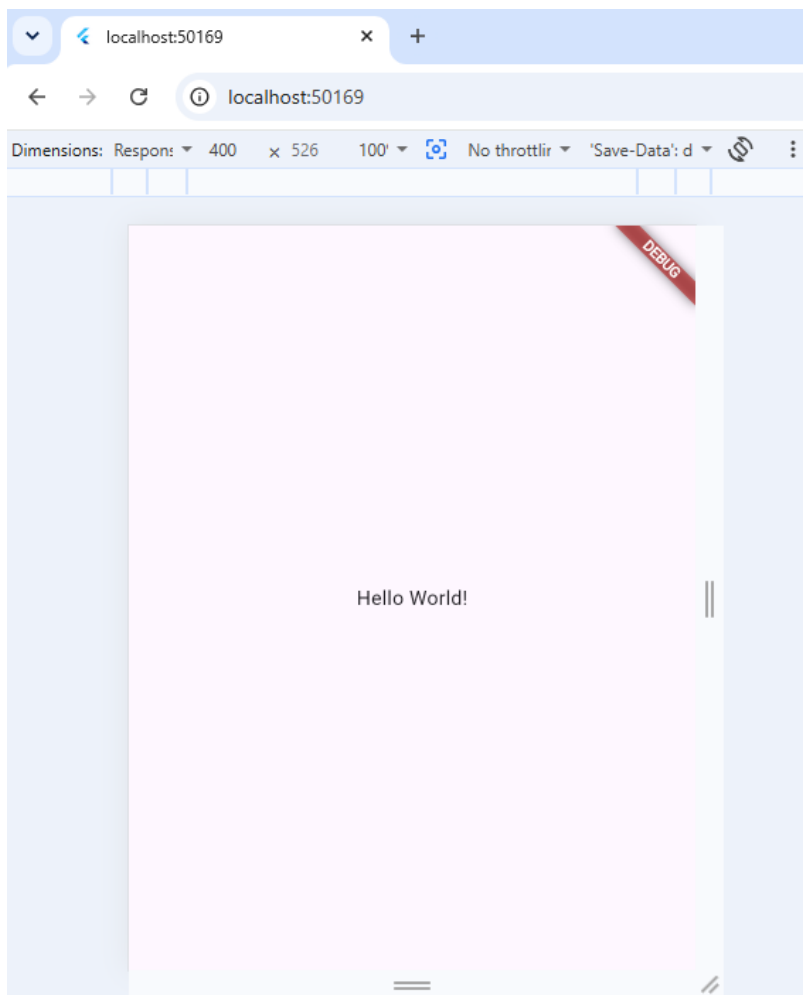
Khi phát triển ứng dụng Flutter, ta thường phải test trên nhiều kích thước màn hình khác nhau (responsive). Có những cách sau hỗ trợ thiết kế responsive:

- Sử dụng chức năng Device Toolbar của trình duyệt (Chrome)
- Sử dụng một Flutter package: device_preview (https://pub.dev/packages/device_preview)
- Sử dụng máy ảo giả lập (Android Emulator / iOS Simulator)

Yêu cầu: Hãy sử dụng chức năng Device Toolbar để xem ứng dụng.

Hướng dẫn:

Nhấp phải vào trình duyệt > bấm Inspect > bấm nút Toogle Device Toolbar.



1.3 Cấu trúc project Flutter

1. Cấu trúc chung

Một Flutter project là một ứng dụng đa nền tảng. Phần code ứng dụng chủ yếu nằm trong lib/, còn các thư mục như android/, ios/, web/, windows/... là phần cấu hình để ứng dụng chạy trên từng nền tảng.

Cấu trúc project Flutter:

```
HELLO_WORLD/  
|  
├── .dart_tool/  
├── .idea/  
|  
├── android/  
├── ios/  
├── lib/  
│   └── main.dart  
|  
├── linux/  
├── macos/  
├── web/  
├── windows/  
|  
├── .gitignore  
├── .metadata  
├── analysis_options.yaml  
├── hello_world.iml  
├── pubspec.lock  
├── pubspec.yaml  
└── README.md
```

Có thể chia thành **3 nhóm**:

1. CODE ỨNG DỤNG

```
|  
└── lib/  
|
```

2. NỀN TẢNG CHẠY ỨNG DỤNG

```
|  
├── android/  
├── ios/  
├── web/  
├── windows/  
├── macos/  
└── linux/  
|
```

3. CẤU HÌNH & QUẢN LÝ PROJECT

```
|  
├── pubspec.yaml  
├── pubspec.lock  
└── analysis_options.yaml
```

```
| .gitignore
| ...
```

Bài tập 8. Tìm hiểu chi tiết về cấu trúc project Flutter

Hãy đọc nội dung bên dưới (hoặc tự nghiên cứu với AI hoặc các tài liệu khác).

Hãy trả lời các câu hỏi sau:

1. “Nếu muốn đổi giao diện màn hình Hello World, ta sửa ở đâu?”
2. “Nếu muốn thêm package vào project?”
3. “Nếu muốn ứng dụng chạy trên Chrome, Flutter sử dụng cấu hình ở thư mục nào?”
4. “Nếu muốn chạy trên Android, Flutter sử dụng cấu hình ở thư mục nào?”
5. “File nào là điểm bắt đầu của chương trình?”

Giới thiệu về cấu trúc project Flutter:

1. Phần code ứng dụng

a) Thư mục lib/

Đây là **thư mục quan trọng nhất đối với lập trình Flutter**.

Sau này project lớn hơn, sẽ gồm có các thư mục con như bên dưới:

```
lib/
| main.dart
| screens/
| widgets/
| models/
| services/
| utils/
```

b) File main.dart

Đây thường là **điểm bắt đầu (entry point)** của ứng dụng Flutter.

Ví dụ:

```
void main() {
  runApp(const MyApp());
}
```

*Hàm main() giống như **cửa chính (entry point)** của ngôi nhà. Khi chạy ứng dụng, chương trình đi vào từ cửa này.*

2. Phần dành cho các nền tảng chạy ứng dụng

a) android/ – phần dành cho Android

Chứa các thành phần cần thiết để Flutter project có thể chạy trên **Android**.

Ví dụ khi chạy:

```
flutter run
```

và chọn **Android Emulator**, Flutter sẽ sử dụng phần Android này để build ứng dụng.

b) ios/ – phần dành cho iOS

Tương tự android/, nhưng dành cho **iOS**.

Thư mục ios/ chứa cấu hình native cần thiết để ứng dụng Flutter chạy trên iPhone/iPad.

Sinh viên làm việc trên Hệ điều hành Windows thường sẽ ít làm việc trực tiếp với thư mục này vì build iOS cần môi trường macOS/Xcode.

c) web/ – chạy ứng dụng trên trình duyệt

Flutter không chỉ chạy trên điện thoại, mà còn có thể chạy:

```
Flutter
├── Android
├── iOS
├── Web
├── Windows
├── macOS
└── Linux
```

Thư mục web/ chứa các thành phần cấu hình cho Flutter Web.

Ví dụ sinh viên có thể chạy:

```
flutter run -d chrome
```

và ứng dụng Flutter sẽ chạy trên Chrome.

d) windows/, macos/, linux/

Ba thư mục này phục vụ việc chạy ứng dụng Flutter trên desktop:

```
windows/ → Windows
macos/   → macOS
```

Điểm hay của Flutter là:

Một codebase có thể phát triển ứng dụng cho nhiều nền tảng.

Phần code Dart/Flutter trong lib/ có thể được sử dụng trên nhiều nền tảng.

3. Phần cấu hình và quản lý project

a) pubspec.yaml – file cực kỳ quan trọng

Đây là file đặc biệt khuyên sinh viên phải biết.

Có thể ví von: **pubspec.yaml giống như “phiếu khai báo” của project.**

Nó mô tả:

- tên project
- version
- môi trường Dart/Flutter
- dependencies
- assets
- fonts
- các package sử dụng

Ví dụ:

```
name: hello_world

description: A Flutter application.

dependencies:
  flutter:
    sdk: flutter

  cupertino_icons: ^1.0.8
```

Khi muốn sử dụng một package bên ngoài, thường sẽ khai báo trong: pubspec.yaml

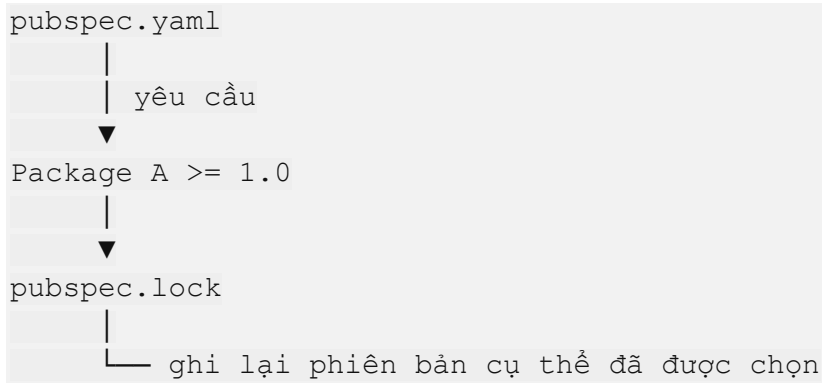
Sau đó chạy lệnh sau để tải package về:

```
flutter pub get
```

b) pubspec.lock – ghi lại phiên bản package thực tế

File: pubspec.lock có liên quan trực tiếp đến pubspec.yaml.

Có thể giải thích:



Ví dụ:

pubspec.yaml → Tôi cần package X

pubspec.lock → Project này hiện đang dùng package X phiên bản 1.2.3

Điều này giúp môi trường của các thành viên trong nhóm ổn định hơn.

c) .dart_tool/ – thư mục sinh ra trong quá trình phát triển

Thư mục .dart_tool/ chứa các thông tin/cấu trúc phục vụ Dart và Flutter tooling.

Chúng ta **không nên tự ý sửa nội dung bên trong**.

Có thể nói đơn giản: Đây là thư mục “hậu trường” do Flutter/Dart tạo ra để hỗ trợ quá trình build và phát triển.

d) .idea/ – cấu hình của IntelliJ/Android Studio

.idea/ chứa các thiết lập liên quan đến IDE JetBrains như:

- Android Studio
- IntelliJ IDEA

Nếu dùng VS Code thì **không cần tập trung vào thư mục này**.

e) File analysis_options.yaml

File: analysis_options.yaml liên quan đến **Dart analyzer và coding rules**.

Ví dụ nó có thể quy định:

- cảnh báo nào được bật
- coding style
- lint rules
- những lỗi cần cảnh báo

Có thể giải thích đơn giản: Đây là nơi quy định “**luật viết code**” cho project.

f) .gitignore

File .gitignore là file này rất quan trọng khi học **Git**.

Nó quy định: Những file/thư mục nào Git **không cần đưa vào repository**.

Ví dụ:

```
.dart_tool/  
build/  
...
```

Thông thường các file sinh ra tự động không cần commit lên Git.

g) .metadata

File .metadata lưu một số metadata liên quan đến Flutter project/tooling.

Khi mới học **không cần chỉnh sửa file này**.

h) File hello_world.iml

hello_world.iml

Đây là file metadata/module của IntelliJ/Android Studio.

Nếu dùng VS Code, chúng ta gần như không cần quan tâm.

i) README.md

README.md là file tài liệu giới thiệu project.

Có thể chứa:

- Tên project

- Mục đích
- Cách cài đặt
- Cách chạy
- Công nghệ sử dụng
- ...

Đặc biệt khi làm project nhóm, nên viết README tốt để thành viên khác có thể clone project và chạy được.

4. Tóm tắt một số thư mục/file quan trọng

Thành phần	Ý nghĩa
lib/	Code ứng dụng
lib/main.dart	Điểm bắt đầu ứng dụng
pubspec.yaml	Khai báo project/package/assets
android/, ios/, web/...	Cấu hình cho từng nền tảng
.gitignore	Quy định file Git bỏ qua

Bài tập 9. Giải thích file main.dart

Hãy đọc nội dung file main.dart và những giải thích bên dưới, sau đó trả lời các câu hỏi sau:

1. Khi chạy chương trình, hàm nào được thực hiện đầu tiên?
2. runApp() có nhiệm vụ gì?
3. Widget gốc trong chương trình này là gì?
4. Hãy vẽ Widget Tree của chương trình.

File main.dart:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
```

```

return const MaterialApp(
  home: Scaffold(
    body: Center(
      child: Text('Hello World!'),
    ),
  ),
);
}

```

Giải thích:

a) Dòng: `import 'package:flutter/material.dart';`

Dòng này giúp đưa thư viện: Flutter Material vào chương trình.

Nhờ đó, chúng ta có thể sử dụng các thành phần như:

- MaterialApp
- Scaffold
- Center
- Text

Nếu bỏ dòng import này, Dart sẽ không biết các class như MaterialApp, Scaffold, Text là gì.

b) Hàm `main()` – điểm bắt đầu của chương trình

Trong Dart: `void main()` là **entry point** – điểm bắt đầu thực thi của chương trình.

Câu lệnh: `runApp()`

Đây là câu lệnh yêu cầu Flutter:

Hãy khởi chạy ứng dụng với Widget MainApp làm Widget gốc.

c) Lớp MainApp là gì?

```

class MainApp extends StatelessWidget {
  . . .
}

```

MainApp là một **Widget** thuộc loại StatelessWidget → giao diện hiển thị.

Widget là gì?

Đây là khái niệm quan trọng nhất của Flutter.

Widget là building block – “viên gạch” để xây dựng giao diện Flutter.

Ví dụ, một số widget phổ biến như:

- Text → hiển thị chữ
- Button → nút
- Image → hình ảnh
- Row → sắp xếp ngang
- Column → sắp xếp dọc
- Center → căn giữa
- Scaffold → cấu trúc màn hình

Vì vậy giao diện Flutter được xây dựng bằng cách **lồng các Widget vào nhau**, tạo thành cây Widget (Widget tree).

d) Hàm build()

```
@override
Widget build(BuildContext context) {
  . . .
}
```

@override có nghĩa là:

Class MainApp đang **ghi đè (override)** một phương thức được định nghĩa ở class cha (StatelessWidget).

StatelessWidget yêu cầu chúng ta triển khai: build()

Vì vậy MainApp phải có: Widget build(BuildContext context)

Hàm build() – trái tim của Widget

Đây là phần **rất quan trọng**.

Phương thức build() có nhiệm vụ: **Mô tả giao diện mà Widget này muốn hiển thị.**

Hay nói đơn giản hơn: Hàm **build()** **trả về Widget Tree** của giao diện.

Tham số BuildContext là gì?

```
build(BuildContext context)
```

context đại diện cho **vị trí của Widget trong Widget Tree**.

Ví dụ:

```
MaterialApp
├── Scaffold
│   └── Center
│       └── Text
```

Mỗi Widget nằm ở một vị trí trong cây.

BuildContext giúp Widget biết được **nó đang ở đâu trong cây Widget** và có thể truy cập một số thông tin liên quan đến vị trí đó.

e) Scaffold là gì?

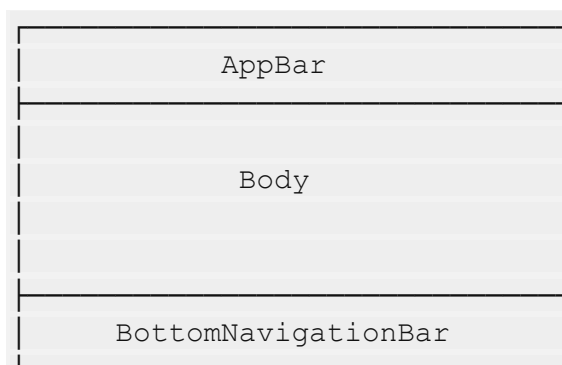
```
Scaffold(  
  body: ...  
)
```

Scaffold cung cấp **bộ khung cơ bản của một màn hình Material Design**.

Ví dụ Scaffold có thể chứa:

```
Scaffold(  
  appBar: ...,  
  body: ...,  
  drawer: ...,  
  floatingActionButton: ...,  
  bottomNavigationBar: ...,  
)
```

Có thể hình dung:



Trong Hello World, chúng ta mới chỉ sử dụng phần body.

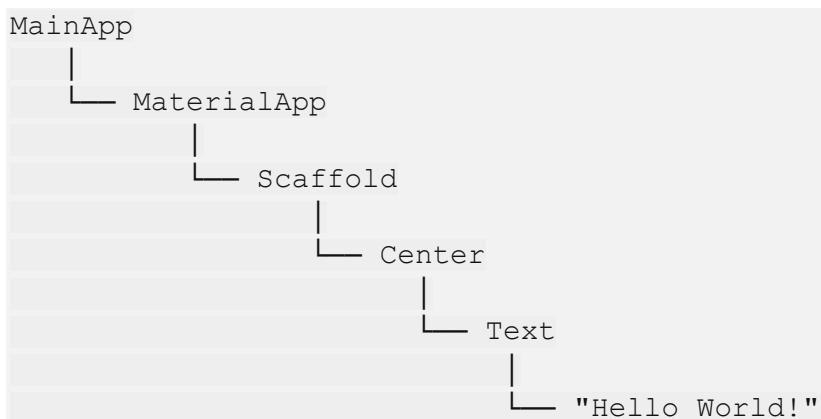
Xem thêm về Scaffold: <https://www.geeksforgeeks.org/flutter/scaffold-class-in-flutter-with-examples/>

f) Các widget Center(), Text()

```
Scaffold(  
  body: Center(  
    child: Text('Hello World!'),  
  ),  
)
```

- Center có nhiệm vụ căn Widget con vào giữa.
- child: thuộc tính của widget Center, chỉ định phần tử con của Center.
- Text('Hello World!'): hiển thị văn bản “Hello World!”

Như vậy toàn bộ giao diện có thể biểu diễn bằng cây:



Đây chính là Widget Tree.

1.4 Một số bài thực hành làm quen với Flutter

Bài tập 10. Áp dụng kiểu (style) cho widget Text

Yêu cầu: Hãy áp dụng kiểu (style) cho chữ “Hello World”: in đậm, màu chữ đỏ, màu nền vàng.

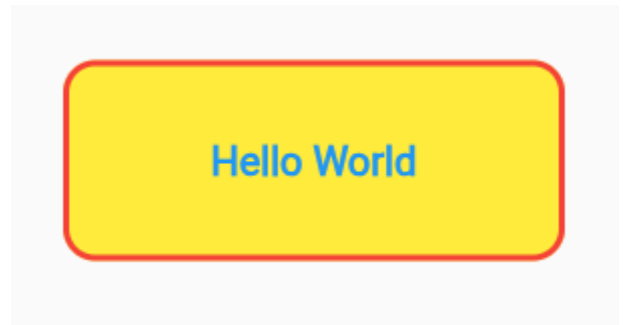
```
import 'package:flutter/material.dart';
void main() => runApp(MyApp());
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Welcome to Flutter',
      home: Scaffold(
        appBar: AppBar(
          title: Text('Welcome to Flutter'),
        ),
        body: Center(
          child: Text(
            "Hello World",
            style: TextStyle(
              fontSize: 24, // cỡ chữ
              fontWeight: FontWeight.bold, // in đậm
              color: Colors.red, // chữ đỏ
              backgroundColor: Colors.yellow, // nền vàng
            ),
          ),
        ),
      ),
    );
  }
}
```

Bài tập 11. Hiện thị hình chữ nhật với widget Container

Yêu cầu:

Hãy sử dụng widget Container, tạo hình chữ nhật như sau:

- Kích thước width: 250px, height: 100px.
- Màu nền: vàng.
- Màu viền: đỏ, độ dày viền: 3px, bo tròn 4 góc.
- Bên trong có chữ "Hello World" được canh giữa (màu chữ: xanh).



```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Container(
          width: 250,
          height: 100,
          decoration: BoxDecoration(
            color: Colors.yellow, // màu nền
            border: Border.all(
              color: Colors.red, // màu viền
              width: 3, // độ dày viền
            ),
            borderRadius: BorderRadius.circular(16), // bo góc
          ),
          alignment: Alignment.center, // căn chữ giữa
          child: const Text(
            "Hello World",
            style: TextStyle(
              fontSize: 20,
              fontWeight: FontWeight.bold,
              color: Colors.blue,
            ),
          ),
        ),
      ),
    );
  }
}
```

Yêu cầu bổ sung: Hãy canh giữa màn hình toàn bộ khối Container.

Bài tập 12. Hiện thị hình tròn với widget Container

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: Container(
            width: 200,
            height: 200,
            decoration: BoxDecoration(
              color: Colors.yellow, // màu nền
              border: Border.all(
                color: Colors.red, // màu viền
                width: 3, // độ dày viền
              ),
              shape: BoxShape.circle, // hình tròn
            ),
            alignment: Alignment.center, // căn chữ vào giữa
            child: const Text(
              "Hello World",
              textAlign: TextAlign.center,
              style: TextStyle(
                fontSize: 20,
                fontWeight: FontWeight.bold,
                color: Colors.red,
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

Bài tập 13. Hiển thị icon

- Đọc thêm về lớp Icon: <https://api.flutter.dev/flutter/material/Icons-class.html>
- Tìm kiếm các icon đẹp trên Google Fonts: <https://material.io/resources/icons>

a) Hiển thị 1 icon



```
import 'package:flutter/material.dart';

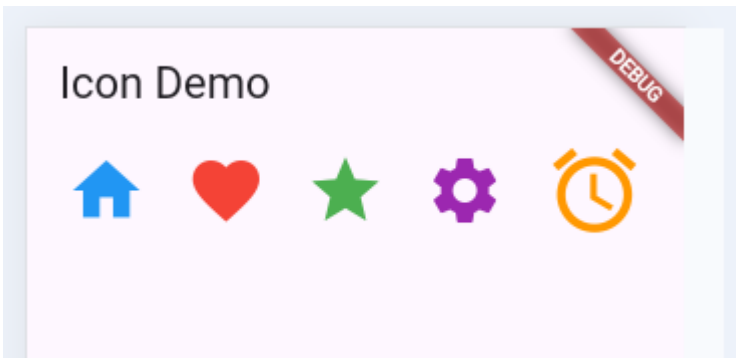
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: Scaffold(
        body: Center(
          child: Icon(
            Icons.favorite, // loại icon
            color: Colors.red, // màu
            size: 80, // kích thước
          ),
        ),
      ),
    );
  }
}
```

b) Hiển thị 5 icon theo chiều ngang

Gợi ý: Dùng widget Row để sắp xếp theo chiều ngang.



```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text("Icon Demo")),
        body: Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          children: const [
            // 1. Icon mặc định
            Icon(
              Icons.home,
              color: Colors.blue,
              size: 40,
            ),

            // 2. Icon màu đỏ
            Icon(
              Icons.favorite,
              color: Colors.red,
              size: 40,
              semanticLabel: "Yêu thích",
            ),

            // 3. Icon màu xanh lá
            Icon(
              Icons.star,
              color: Colors.green,
              size: 40,
            ),
          ],
        ),
      ),
    );
  }
}
```

```
// 4. Icon màu tím với bóng
Icon(
  Icons.settings,
  color: Colors.purple,
  size: 40,
),

// 5. Icon màu cam, lớn hơn
Icon(
  Icons.alarm,
  color: Colors.orange,
  size: 50,
),
],
),
),
);
}
```

Bài tập 14. Hiển thị ảnh từ asset với Image.asset (ảnh được đóng gói trong app)

Các bước thực hiện:

Bước 1: Đặt ảnh vào thư mục assets/ trong project Flutter của bạn.

Tạo thư mục `assets/images` và copy ảnh vào thư mục này.

Ví dụ: `assets/images/flutter_logo.png`

Bước 2: Khai báo trong file pubspec.yaml:

```
flutter:  
  assets:  
    - assets/images/flutter_logo.png
```

Chú ý: có 2 khoảng trắng thật đầu dòng thể hiện sự phân cấp flutter → assets → link đến ảnh.

Bước 3: Trong file main.dart, dùng `Image.asset` để hiển thị ảnh:

```
import 'package:flutter/material.dart';  
  
void main() {  
  runApp(const MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        body: Center(  
          child: Image.asset(  
            'assets/images/flutter_logo.png', // đường dẫn tới ảnh  
            width: 150, // chiều rộng  
            height: 150, // chiều cao  
            fit: BoxFit.contain, // cách hiển thị trong khung  
          ),  
        ),  
      ),  
    );  
  }  
}
```

Kết quả: Ảnh Flutter logo sẽ hiển thị ở giữa màn hình, với kích thước 150x150.

Chú ý: Ta cần Stop và Run ứng dụng lại để thấy hình ảnh hiển thị trên ứng dụng.

Bài tập 15. Hiển thị ảnh từ internet với Image.network

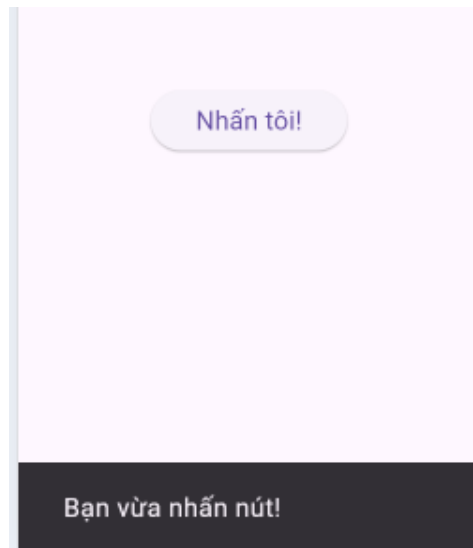
```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: Image.network(
            'https://flutter.github.io/assets-for-api-docs/assets/widgets/owl.jpg',
            width: 200, // chiều rộng
            height: 200, // chiều cao
            fit: BoxFit.cover, // hiển thị ảnh đầy khung
          ),
        ),
      ),
    );
  }
}
```

Bài tập 16. Hiển thị một nút (button) trong Flutter



```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: ElevatedButton(
            onPressed: () {
              // Khi nhấn nút, hiển thị Snackbar
              ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(content: Text('Bạn vừa nhấn nút!')),
              );

              // Đồng thời in ra console (Debug Console)
              print('Button pressed!');
            },
            child: const Text('Nhấn tôi!'),
          ),
        ),
      ),
    );
  }
}
```

Bài tập 17. Tạo bộ khung cho màn hình với widget Scaffold

Scaffold là Widget cung cấp **bộ khung cơ bản** cho một màn hình Flutter.

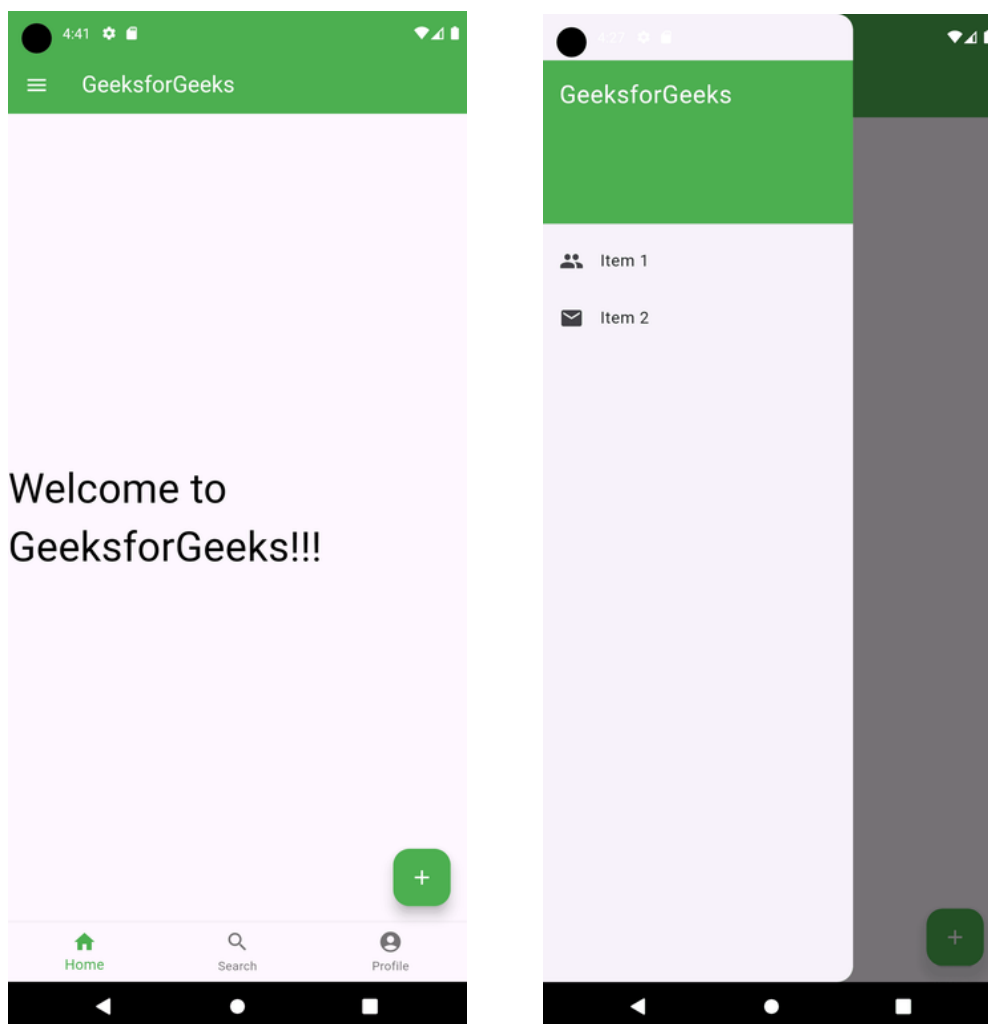
Nó giúp tổ chức các thành phần như:

```
Scaffold(  
  appBar: AppBar(),           // Thanh tiêu đề  
  body: ...,                  // Nội dung chính  
  drawer: ...,                // Menu bên  
  floatingActionButton: ...,   // Nút nổi  
  bottomNavigationBar: ...,   // Thanh điều hướng dưới  
)
```

Yêu cầu:

Hãy đọc về Scaffold và thực hiện các bài tập tại link này:

<https://www.geeksforgeeks.org/flutter/scaffold-class-in-flutter-with-examples/>



1.5 Troubleshooting

1.5.1 Error: Null safety features are disabled for this library

Tham khảo: <https://fluttercorner.com/error-null-safety-features-are-disabled-for-this-library/>

1. Open **your terminal** in root level of your project.
2. run **flutter upgrade**.
3. Change SDK version in pubspec.yaml file
4. In my pubspec.yaml I just changed This line **sdk: “>=2.7.0 <3.0.0”** and replaced with **sdk: “>=2.12.0 <3.0.0”**
5. flutter clean
6. flutter run.